

Section 10.2: Site Reliability Engineering (SRE)

Principles

*Study Guide | 3 Files in This Series | Section 10.**

Executive Overview

SRE is the discipline of applying software engineering to operations problems. Its core tension is between **reliability** (keeping things running) and **feature velocity** (shipping new things). Error budgets are the mechanism that makes this tension explicit and manageable. This section covers SLOs, burn rates, toil reduction, and incident management — the four pillars of SRE practice.

1. Service Level Objectives (SLOs) & Error Budgets

Defining SLOs

An SLO is a target reliability level expressed as a ratio over a time window. It converts a vague goal (“the service should be fast”) into a quantified, trackable commitment.

Common SLO types:

Availability:	99.9% of requests succeed	→ 43 min downtime/month
Latency:	95th percentile response time < 200ms	
Freshness:	Data updated within 5 minutes of source	
Correctness:	99.99% of calculations return right result	

Error budget derivation (worked example):

```
SLO = 99.9% → allowed failure rate = 0.1%
Window = 30 days = 43,200 minutes

Error budget = 0.1% × 43,200 min = 43.2 minutes of allowable downtime

At 1,000 requests/minute:
```

```
Total requests = 43,200,000
Allowed failures = 0.1% × 43,200,000 = 43,200 failed requests
```

Key insight: The error budget is not a punishment threshold — it is a *resource* that teams spend on risk. Deploying a risky change consumes budget. Running chaos experiments consumes budget. Keeping everything frozen conserves budget but starves innovation.

Burn Rate — Measuring Consumption Velocity

Burn rate expresses how fast you are consuming the error budget relative to a sustainable pace.

```
Burn rate = 1.0 → consuming budget exactly on pace (uses 100% in 30 days)
Burn rate = 2.0 → consuming 2× as fast (budget exhausted in 15 days)
Burn rate = 12.0 → budget exhausted in 2.5 days — page immediately
```

Burn rate worked example:

```
SLO window: 30 days
Observed: 2% of budget consumed in 1 hour

Burn rate = (2% / 100%) × (30 days × 24 hr/day) / 1 hr
           = 0.02 × 720 / 1
           = 14.4×

Interpretation: At this rate, the budget is gone in 30/14.4 ≈ 2 days.
Action: Page on-call immediately; halt releases.
```

Alert Tiers Based on Burn Rate

Burn rate	Budget consumed	Time to exhaustion	Action
> 14×	2% in 1 hr	~2 days	Page immediately
> 6×	5% in 6 hr	~5 days	Page within 30 min
> 3×	10% in 24 hr	~10 days	Ticket + monitor
≤ 1×	On pace	30 days	No action

Release Management via Error Budget

```
class ReleaseManager:
    def should_block_release(self) -> tuple[bool, str]:
        burn_rate = self.calculate_burn_rate()
```

```
    if burn_rate > 2.0:
        return True, f"Fast burn at {burn_rate:.1f}x – stabilise first"

    if self.error_budget_remaining() < 0.10:
        return True, f"Only {self.error_budget_remaining():.1%} budget
remains"

    return False, "Budget healthy – safe to release"
```

Policy logic: When budget is healthy, teams can ship aggressively. When budget is low, reliability work takes priority over features. This is a *negotiated agreement* between product and engineering, not an engineering veto.

2. Toil Reduction

Identifying Toil

Toil is operational work that is **manual, repetitive, automatable, and produces no lasting value**. It is distinct from overhead (meetings, planning) and engineering (building automation).

Classic toil examples: - Manually restarting a service after memory leak crashes - Rolling back a deploy via a Confluence runbook with 20 steps - Rotating log files via SSH on each host - Manually scaling a deployment during a traffic spike

Test for toil: Would a script do this just as well? If yes, it is toil.

The 50% Rule

SRE teams commit to spending at most 50% of their time on operational (toil) work. If operational work exceeds 50%, it is escalated to management as a capacity issue — not accepted as normal.

```
Time budget:
  ≤ 50% → Incidents, on-call, manual ops
  ≥ 50% → Automation, architecture, monitoring improvements
```

This ratio protects against the “ops death spiral” where incident response crowds out the automation work that would reduce future incidents.

Building Idempotent Automation

All automation must be **safe to re-run** without side effects:

```
def scale_service(service_name: str, target_replicas: int) -> str:
    """Safe, idempotent scaling – can be called repeatedly."""

    current = kubernetes.get_replicas(service_name)

    if current == target_replicas:
        return f"No-op: already at {target_replicas}"

    kubernetes.scale_deployment(service_name, target_replicas)
    wait_for_stable(service_name, timeout=300)

    actual = kubernetes.get_replicas(service_name)
    assert actual == target_replicas, "Scale did not converge"

    return f"Scaled {service_name}: {current} → {target_replicas}"
```

Why idempotency matters: Automation runs in uncertain environments. Network timeouts, retries, and concurrent calls are common. Idempotent operations are safe regardless of how many times they execute.

3. Incident Management

Blameless Post-Mortem Culture

A blameless post-mortem assumes that engineers made reasonable decisions given the information available at the time. The goal is to find *system* weaknesses, not blame individuals.

Timeline reconstruction (example):

```
14:00:32  Deploy starts (v1.2.3)
14:02:15  Error rate spikes to 50%
14:03:45  PagerDuty fires
14:05:12  Rollback initiated by on-call
14:15:00  Service restored
14:20:00  Root cause identified: DB connection pool exhausted
```

Root cause vs contributing factors:

```
Root cause:
    Database connection pool exhausted under new query pattern

Contributing causes (each must be fixed independently):
    1. No alert on connection pool saturation (monitoring gap)
    2. New query lacked index – full table scan × 10 replicas
```

3. Auto-scaling excluded the database tier
4. Integration test suite missed connection exhaustion edge case

Fixing only the root cause without addressing contributing causes means the *next* incident will have different surface symptoms but the same underlying fragility.

Action Item Priority Framework

P0 – Fix now (prevents recurrence immediately):
Add DB pool saturation alert

P1 – Fix this sprint:
Add slow-query logging; add the missing index

P2 – Fix this quarter:
Update runbook with DB rollback steps

P3 – Backlog with owner:
Add integration test for connection exhaustion

All items go into a tracking system (Jira, Linear) with an owner and due date. Untracked action items disappear within two weeks.

4. SLO Approaches – Comparison

Dimension	Strict SLO (99.99%)	Relaxed SLO (99.5%)
Error budget	4 min/month	3.6 hr/month
Release velocity	Very constrained	Relatively free
Engineering investment	High (redundancy, DR)	Moderate
Best for	Payment processing, auth	Internal tools, analytics

How to choose: Anchor on user impact. Ask: “If this is down for 1 hour, what is the business cost?”
Payment processing: very high → 99.99%. Internal reporting: low → 99.5%.

5. Interview Problems

Problem 1 — 80% Error Budget Burned in 6 Hours

Scenario: Your service has a 99.9% SLO. During an incident, 80% of the monthly error budget was consumed in 6 hours. What do you do, and what does this reveal?

Step 1 — Quantify burn rate:

```
Burn rate = (fraction_consumed / total_budget_fraction) × (window_hours /  
observation_hours)  
           = (0.80 / 1.0) × (30 × 24 / 6)  
           = 0.80 × 120  
           = 96×
```

Normal sustainable rate = 1×
Current rate = 96× normal

At this rate, remaining 20% budget exhausted in:
 $(0.20 / 0.80) × 6 \text{ hours} = 1.5 \text{ hours}$

Step 2 — Immediate response:

1. Halt all non-critical releases (budget policy triggered)
2. Engage on-call escalation chain
3. Notify stakeholders: estimated impact, ETA
4. Activate traffic shedding if service is still degraded:
 - Disable non-critical features (recommendations, analytics)
 - Rate-limit burst traffic
 - Serve cached responses where tolerable

Step 3 — What this reveals about the system:

The 96× burn rate points to at least three systemic weaknesses:

Observation	Implication
No early warning before 80% consumed	Alerting fires too late; need multi-window burn rate alerts
80% in 6 hours possible	Single failure mode affects the entire SLO window
Error budget exhausted mid-month	Feature velocity was too high relative to reliability investment

Step 4 — Prevention engineering:

```
# Before: No headroom
# Current peak utilisation: 85%
# After: Target 50% with 2× capacity headroom

# Add circuit breaker on high-risk dependencies
def with_circuit_breaker(func):
    if circuit.state == "OPEN":
        return fallback_response()
    return func()

# Add multi-window burn rate alert (see alerting table above)
```

Key insight: A single fast-burn incident consuming most of the monthly budget is a signal that reliability is under-invested. The error budget framework surfaces this explicitly — use the data to drive a reliability sprint, not just an incident retrospective.

Problem 2 — Toil Has Exceeded 50%

Scenario: Your SRE team tracks their time and discovers 70% is spent on operational work — mostly manual certificate rotations and deployment rollbacks. You have a new headcount request pending but leadership wants evidence first.

Solution:

Step 1 — Quantify toil with data:

```
# Time-tracking export from PagerDuty + Jira
toil_breakdown = {
    "certificate_rotation": "18 hrs/month (6 certs × 3 hrs each)",
    "manual_rollbacks":    "12 hrs/month (4 rollbacks × 3 hrs each)",
    "log_rotation":        "8 hrs/month",
    "manual_scaling":      "6 hrs/month",
    "total":               "44 hrs/month of 63hr capacity = 70%"
}
```

Step 2 — Automation ROI argument:

```
Certificate auto-rotation (cert-manager in Kubernetes):
Investment: 1 sprint (2 engineers × 2 weeks) = 80 engineer-hours
Savings: 18 hrs/month reclaimed
Payback period: 80 / 18 ≈ 4.4 months

Automated rollback on error-rate spike:
Investment: 1 sprint = 80 engineer-hours
```

Savings: 12 hrs/month
Payback period: 6.7 months

Step 3 — Present as a reliability investment, not a staffing request: - Current state: 70% toil → 30% engineering → automation compounds slowly - Target state: 40% toil → 60% engineering → faster compounding of automation - New headcount buys runway to build automation that permanently reduces toil

Problem 3 — Designing an SLO for a New Service

Scenario: You are launching a new checkout API. Product wants 99.99%. Engineering says 99.9% is achievable on the current architecture. How do you resolve this?

Solution framework:

Step 1 — Map SLO to user-facing impact:

99.99% → 4.4 min downtime/month → user almost never sees an error
99.9% → 43 min downtime/month → ~1 incident/month lasting ~40 min
99.5% → 3.6 hr downtime/month → noticeable to regular customers

Step 2 — Estimate architectural cost of closing the gap:

99.9% (current):
Single-region active-passive DB
Single-AZ application tier
Cost: baseline

99.99% (product request):
Multi-region active-active
Synchronous DB replication
Global load balancer
Additional cost: ~4× infrastructure, 2 sprints of engineering

Step 3 — Propose a graduated commitment:

Month 1-3: SLO = 99.5% (prove the baseline works)
Month 4-6: SLO = 99.9% (add DR, fix monitoring gaps)
Month 7+: Re-evaluate 99.99% based on actual burn rate data

Key insight: An SLO is a contract. Committing to 99.99% without the architecture to support it creates a reliability debt that manifests as incident pressure on the SRE team. Start with what the system can actually deliver and improve iteratively.

Quick-Reference Cheat Sheet

Error budget = $(1 - \text{SLO}) \times \text{window}$
Burn rate = $(\text{budget_consumed} / \text{window})$ normalised to 1.0
Fast burn ($>14\times$) → page; Slow burn ($3-6\times$) → ticket; $\leq 1\times$ → normal
50% rule → $>50\%$ toil = escalate to management, not absorb
Post-mortem → blameless; root cause + contributing causes + owners
SLO selection → anchor on business cost of downtime, not gut feeling